# IPLD 데이터 모델 및 DAG 구조

## 개요

IPLD(InterPlanetary Linked Data)는 IPFS의 핵심 데이터 모델로, 다양한 데이터 형식을 통합된 방식으로 표현하고 연결할 수 있게 해주는 시스템입니다. IPLD는 DAG(Directed Acyclic Graph) 구조를 사용하여 데이터 간의 관계를 표현하며, 콘텐츠 주소 지정을 통해 데이터의 무결성과 중복 제거를 보장합니다.

## IPLD 데이터 모델

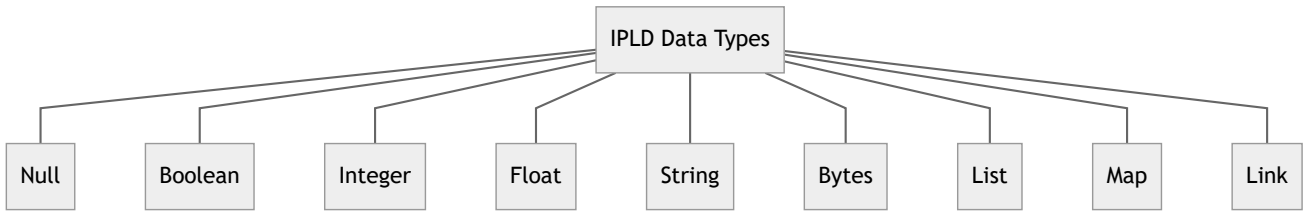
### 기본 개념

IPLD는 다음과 같은 핵심 개념들을 기반으로 합니다:

1. **노드(Node)**: 데이터의 기본 단위
2. **링크(Link)**: 노드 간의 연결
3. **경로(Path)**: 노드 내의 특정 데이터에 접근하는 방법
4. **코덱(Codec)**: 데이터 직렬화/역직렬화 방식

### IPLD 데이터 타입

IPLD는 다음과 같은 기본 데이터 타입을 지원합니다:



### 기본 타입 설명

1. **Null**: 빈 값
2. **Boolean**: 참/거짓 값
3. **Integer**: 정수 값
4. **Float**: 부동소수점 값
5. **String**: 문자열
6. **Bytes**: 바이트 배열
7. **List**: 순서가 있는 값들의 배열
8. **Map**: 키-값 쌍의 집합
9. **Link**: 다른 IPLD 노드에 대한 참조 (CID)

## 노드 구조

IPLD 노드는 다음과 같은 구조를 가집니다:

```
type Node interface {
Kind() Kind // 노드의 타입
LookupByString(key  string) (Node, error) // 문자열 키로 조회
LookupByIndex(idx  int64) (Node, error) // 인덱스로 조회
LookupBySegment(seg PathSegment) (Node, error) // 경로 세그먼트로 조회
MapIterator() MapIterator // 맵 반복자
ListIterator() ListIterator // 리스트 반복자
Length() int64  // 길이
IsAbsent() bool  // 값 부재 여부
IsNull() bool  // null 여부
```

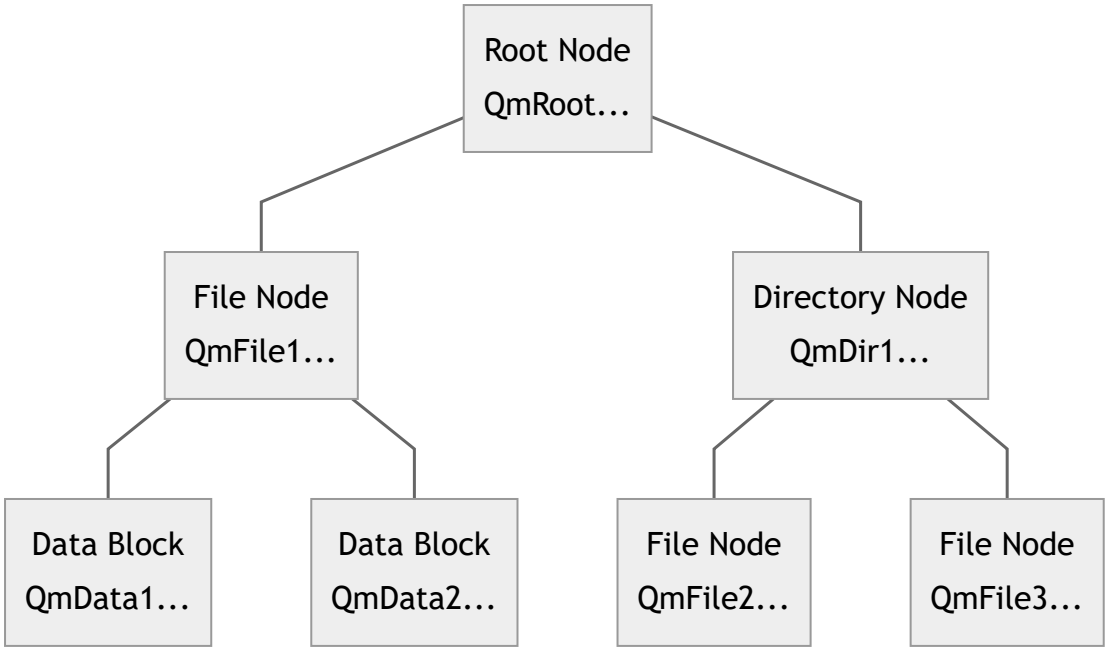
```
AsBool() (bool, error) // 불린 값으로 변환

AsInt() (int64, error) // 정수 값으로 변환

AsFloat() (float64, error) // 실수 값으로 변환

AsString() (string, error) // 문자열 값으로 변환

AsBytes() ([]byte, error) // 바이트 배열로 변환

AsLink() (Link, error) // 링크로 변환

}
```

# DAG (Directed Acyclic Graph) 구조

## DAG의 특성

1. **방향성(Directed)**: 링크는 방향을 가짐
2. **비순환성(Acyclic)**: 순환 참조가 없음
3. **연결성**: 노드들이 링크로 연결됨

## DAG 구조 예시




## 노드 간 연결 방식

### 링크 구조

```
type Link struct {
Name  string  // 링크 이름 (선택적)
Cid cid.Cid // 대상 노드의 CID
Size  uint64  // 대상 노드의 크기 (선택적)
}
```

### 링크 해결 과정

1. **CID 추출**: 링크에서 대상 노드의 CID 추출
2. **노드 검색**: 블록스토어에서 CID로 노드 검색
3. **역직렬화**: 노드 데이터를 IPLD 노드로 역직렬화
4. **경로 탐색**: 필요시 노드 내부 경로 탐색

# 주요 IPLD 코덱

## 1. DAG-PB (Protocol Buffers)

**특징:**

- IPFS의 기본 코덱
- UnixFS에서 사용
- 효율적인 바이너리 형식

**구조:**

```
message PBNode {
repeated PBLink Links = 2;
optional bytes Data = 1;
}

message PBLink {
optional bytes Hash = 1;
optional string Name = 2;
optional uint64 Tsize = 3;
}
```

**사용 예시:**

```
// DAG-PB 노드 생성
node := &dag.ProtoNode{
```

```
Links: []*ipld.Link{
{
Name: "file1.txt",
Cid: fileCid,
Size: 1024,
},
},
Data: []byte("directory metadata"),
}
```

## 2. DAG-CBOR (Concise Binary Object Representation)

**특징:**

- 구조화된 데이터에 최적화
- JSON과 유사한 구조
- 효율적인 바이너리 인코딩

**사용 예시:**

```
// DAG-CBOR 데이터 구조
data := map[string]interface{}{
"name": "example",
"links": []cid.Cid{
childCid1,
childCid2,
```

```
},
"metadata": map[string]interface{}{
"created": time.Now(),
"size": 2048,
},
}
```

## 3. DAG-JSON

**특징:**

- 사람이 읽기 쉬운 형식
- 디버깅과 개발에 유용
- JSON과 호환

**사용 예시:**

```
{
"name": "example",
"links": [
{"/": "QmExample1..."},
{"/": "QmExample2..."}
],
"metadata": {
"created": "2023-01-01T00:00:00Z",
"size": 2048
```

```
}

}
```

### 4. Raw

**특징:**

- 원시 바이트 데이터
- 추가 구조 없음
- 파일 데이터 블록에 사용

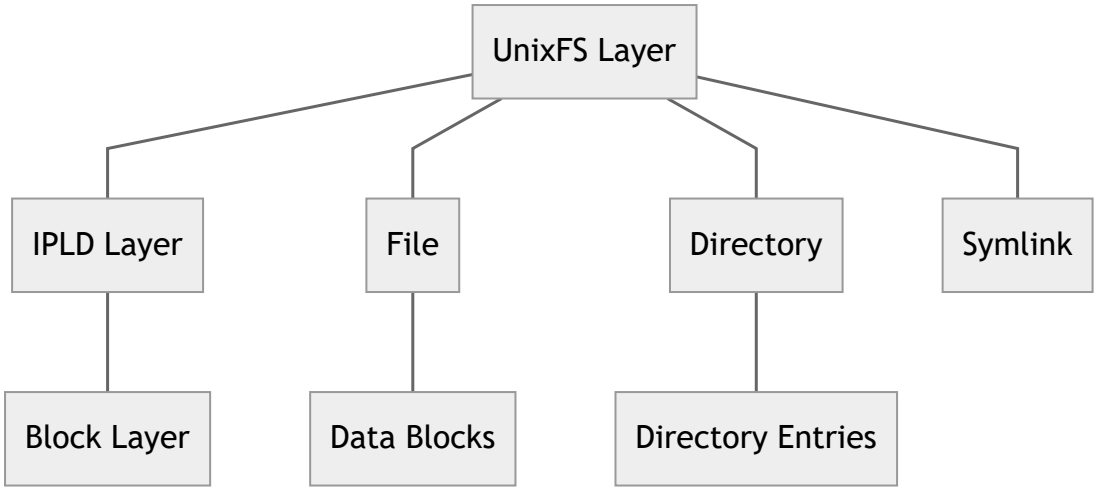
**사용 예시:**

```
// Raw 데이터 블록
rawData := []byte("This is raw file content")
rawCid := cid.NewCidV1(cid.Raw, multihash.Sum(rawData))
```

## UnixFS와 IPLD

### UnixFS 구조

UnixFS는 IPLD 위에 구축된 파일 시스템 추상화입니다:




## UnixFS 노드 타입

### 1. 파일 노드

```
type UnixFSFile struct {
Type pb.Data_DataType // 파일 타입
Data []byte  // 파일 데이터 (작은 파일의 경우)
Filesize  uint64  // 전체 파일 크기
Links []*ipld.Link // 데이터 블록 링크들
}
```

### 2. 디렉토리 노드

```
type UnixFSDirectory struct {
Type pb.Data_DataType // 디렉토리 타입
Links []*ipld.Link // 하위 항목들에 대한 링크
Fanout  uint64  // HAMT 팬아웃 (큰 디렉토리의 경우)
```

```
}
```

## 파일 청킹 전략

### 고정 크기 청킹

```
// 기본 청크 크기: 256KB
const DefaultChunkSize = 262144

func ChunkFile(data []byte) [][]byte {
var chunks [][]byte
for i := 0; i < len(data); i += DefaultChunkSize {
end := i + DefaultChunkSize
if end > len(data) {
end = len(data)
}
chunks = append(chunks, data[i:end])
}
return chunks
}
```

### 가변 크기 청킹 (Rabin)

```
// Rabin 청킹을 사용한 중복 제거 최적화
func RabinChunk(data []byte, avgSize int) [][]byte {
```

```
// Rabin 핑거프린팅을 사용하여
// 콘텐츠 기반 경계 결정
// 중복 제거 효율성 향상
}
```

# DAG 탐색 및 조작

## 경로 기반 탐색

```
func TraversePath(ctx context.Context, dag ipld.DAGService, root cid.Cid, path
// 루트 노드 가져오기
rootNode, err := dag.Get(ctx, root)
if err != nil {
return nil, err
}
// 경로 파싱
segments := strings.Split(path, "/")
currentNode := rootNode
// 각 경로 세그먼트 탐색
for _, segment := range segments {
if segment == "" {
continue
}
```

```
// 링크 찾기
link, err := currentNode.GetNodeLink(segment)
if err != nil {
return nil, err
}
// 다음 노드 가져오기
currentNode, err = dag.Get(ctx, link.Cid)
if err != nil {
return nil, err
}
}
return currentNode, nil
}
```

## DAG 순회

```
func WalkDAG(ctx context.Context, dag ipld.DAGService, root cid.Cid, visitor f
visited := make(map[cid.Cid]bool)
var walk func(cid.Cid) error
walk = func(c cid.Cid) error {
// 이미 방문한 노드는 스킵
if visited[c] {
```

```
return nil
}
visited[c] = true
// 노드 가져오기
node, err := dag.Get(ctx, c)
if err != nil {
return err
}
// 방문자 함수 호출
if err := visitor(node); err != nil {
return err
}
// 모든 링크 순회
for _, link := range node.Links() {
if err := walk(link.Cid); err != nil {
return err
}
}
return nil
}
return walk(root)
}
```

# DAG 조작 명령어

## DAG Put

```
func DagPut(data []byte, inputCodec, storeCodec string) (cid.Cid, error) {
// 입력 코덱으로 디코딩
decoder, err := multicodec.LookupDecoder(inputCodec)
if err != nil {
return cid.Undef, err
}
// 노드 빌더 생성
nodeBuilder := basicnode.Prototype.Any.NewBuilder()
if err := decoder(nodeBuilder, bytes.NewReader(data)); err != nil {
return cid.Undef, err
}
node := nodeBuilder.Build()
// 저장 코덱으로 인코딩
encoder, err := multicodec.LookupEncoder(storeCodec)
if err != nil {
return cid.Undef, err
}
var buf bytes.Buffer
if err := encoder(node, &buf); err != nil {
```

```
return cid.Undef, err
}
// CID 생성
cidPrefix := cid.Prefix{
Version: 1,
Codec: getCodecCode(storeCodec),
MhType: multihash.SHA2_256,
MhLength: -1,
}
return cidPrefix.Sum(buf.Bytes())
}
```

## DAG Get

```
func DagGet(c cid.Cid, outputCodec string) ([]byte, error) {
// 블록 가져오기
block, err := blockstore.Get(c)
if err != nil {
return nil, err
}
// 입력 코덱으로 디코딩
inputCodec := getCodecFromCid(c)
decoder, err := multicodec.LookupDecoder(inputCodec)
```

```
if err != nil {
return nil, err
}
nodeBuilder := basicnode.Prototype.Any.NewBuilder()
if err := decoder(nodeBuilder, bytes.NewReader(block.RawData())); err != nil {
return nil, err
}
node := nodeBuilder.Build()
// 출력 코덱으로 인코딩
encoder, err := multicodec.LookupEncoder(outputCodec)
if err != nil {
return nil, err
}
var buf bytes.Buffer
if err := encoder(node, &buf); err != nil {
return nil, err
}
return buf.Bytes(), nil
}
```

# 성능 최적화

## 배치 처리

```go
type BatchAdder struct {
dag ipld.DAGService
batch ipld.Batch
}

func (b *BatchAdder) Add(ctx context.Context, node ipld.Node) error {
return b.batch.Add(ctx, node)
}

func (b *BatchAdder) Commit() error {
return b.batch.Commit()
}
```

### 캐싱 전략

1. **노드 캐싱**: 자주 접근되는 노드를 메모리에 캐시
2. **링크 캐싱**: 링크 해결 결과 캐시
3. **경로 캐싱**: 경로 탐색 결과 캐시

### 병렬 처리

```go
func ParallelDAGWalk(ctx context.Context, dag ipld.DAGService, roots []cid.Cid,
semaphore := make(chan struct{}, 10) // 동시 실행 제한
var wg sync.WaitGroup
```

```
for _, root := range roots {
wg.Add(1)
go func(c cid.Cid) {
defer wg.Done()
semaphore <- struct{}{}
defer func() { <-semaphore }()
WalkDAG(ctx, dag, c, visitor)
}(root)
}
wg.Wait()
return nil
}
```

## 데이터 무결성 및 검증

### 링크 검증

```
func VerifyDAGIntegrity(ctx context.Context, dag ipld.DAGService, root cid.Cid)
return WalkDAG(ctx, dag, root, func(node ipld.Node) error {
// 각 링크 검증
for _, link := range node.Links() {
// 링크된 노드가 존재하는지 확인
_, err := dag.Get(ctx, link.Cid)
```

```
if err != nil {
return fmt.Errorf("broken link to %s: %w", link.Cid, err)
}
}
return nil
})
}
```

## 구조 검증

```
func ValidateUnixFSStructure(node ipld.Node) error {
// UnixFS 메타데이터 검증
unixfsData, err := unixfs.FSNodeFromBytes(node.Data())
if err != nil {
return err
}
// 타입별 검증
switch unixfsData.Type() {
case unixfs.TFile:
return validateFileStructure(node, unixfsData)
case unixfs.TDirectory:
return validateDirectoryStructure(node, unixfsData)
default:
```

```
return  fmt.Errorf("unknown UnixFS type: %v", unixfsData.Type())

}

}
```

이러한 IPLD 데이터 모델과 DAG 구조를 통해 IPFS는 다양한 데이터 형식을 통합된 방식으로 저장하고 관리하며, 효율적인 데이터 연결과 탐색을 제공합니다.